In [ ]:
```
#importing libraries and dataset
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras import Model, Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.model_selection import train_test_split
from tensorflow.keras.losses import MeanSquaredLogarithmicError

PATH_TO_DATA = 'http://storage.googleapis.com/download.tensorflow.org/dat
data = pd.read_csv(PATH_TO_DATA, header=None)
data.head()
```

In [2]:
```
#finding shape of the dataset
data.shape
```

Out[2]: (4998, 141)

In [3]:
```
#splitting training and testing dataset
features = data.drop(140, axis=1)
target = data[140]
x_train, x_test, y_train, y_test = train_test_split(
    features, target, test_size=0.2, stratify=target
)
train_index = y_train[y_train == 1].index
train_data = x_train.loc[train_index]
```

In [4]:
```
#scaling the data using MinMaxScaler
min_max_scaler = MinMaxScaler(feature_range=(0, 1))
x_train_scaled = min_max_scaler.fit_transform(train_data.copy())
x_test_scaled = min_max_scaler.transform(x_test.copy())
```

In [5]:
```
#creating autoencoder subclass by extending Model class from keras
class AutoEncoder(Model):
  def __init__(self, output_units, ldim=8):
    super().__init__()
    self.encoder = Sequential([
      Dense(64, activation='relu'),
      Dropout(0.1),
      Dense(32, activation='relu'),
      Dropout(0.1),
      Dense(16, activation='relu'),
      Dropout(0.1),
      Dense(ldim, activation='relu')
    ])
    self.decoder = Sequential([
      Dense(16, activation='relu'),
      Dropout(0.1),
      Dense(32, activation='relu'),
      Dropout(0.1),
      Dense(64, activation='relu'),
      Dropout(0.1),
      Dense(output_units, activation='sigmoid')
```

```
    ])

  def call(self, inputs):
    encoded = self.encoder(inputs)
    decoded = self.decoder(encoded)
    return decoded
```

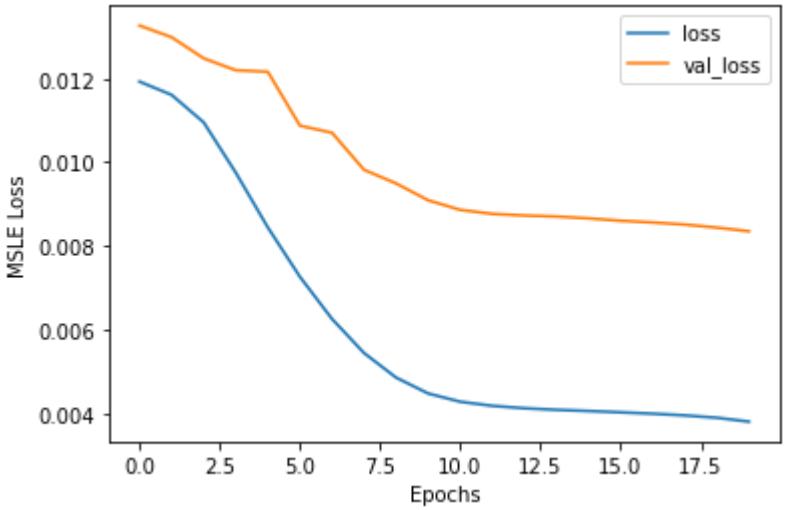
In [6]:
```
#model configuration
model = AutoEncoder(output_units=x_train_scaled.shape[1])
model.compile(loss='msle', metrics=['mse'], optimizer='adam')
epochs = 20

history = model.fit(
    x_train_scaled,
    x_train_scaled,
    epochs=epochs,
    batch_size=512,
    validation_data=(x_test_scaled, x_test_scaled)
)
```

```
Epoch 1/20
E0000 00:00:1793419282.120837    6152 cuda_platform.cc:52] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - loss: 0.0119 - mse: 0.0261 - val_l
oss: 0.0133 - val_mse: 0.0303
Epoch 2/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0116 - mse: 0.0254 - val_l
oss: 0.0130 - val_mse: 0.0297
Epoch 3/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0110 - mse: 0.0240 - val_l
oss: 0.0125 - val_mse: 0.0285
Epoch 4/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0098 - mse: 0.0213 - val_l
oss: 0.0122 - val_mse: 0.0278
Epoch 5/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0084 - mse: 0.0184 - val_l
oss: 0.0122 - val_mse: 0.0276
Epoch 6/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0073 - mse: 0.0158 - val_l
oss: 0.0109 - val_mse: 0.0249
Epoch 7/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0063 - mse: 0.0137 - val_l
oss: 0.0107 - val_mse: 0.0245
Epoch 8/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0054 - mse: 0.0119 - val_l
oss: 0.0098 - val_mse: 0.0226
Epoch 9/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0049 - mse: 0.0107 - val_l
oss: 0.0095 - val_mse: 0.0220
Epoch 10/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0045 - mse: 0.0099 - val_l
oss: 0.0091 - val_mse: 0.0211
Epoch 11/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0043 - mse: 0.0095 - val_l
oss: 0.0089 - val_mse: 0.0207
Epoch 12/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0042 - mse: 0.0093 - val_l
oss: 0.0088 - val_mse: 0.0205
Epoch 13/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0041 - mse: 0.0092 - val_l
oss: 0.0087 - val_mse: 0.0204
Epoch 14/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0041 - mse: 0.0091 - val_l
oss: 0.0087 - val_mse: 0.0204
Epoch 15/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0041 - mse: 0.0090 - val_l
oss: 0.0087 - val_mse: 0.0203
Epoch 16/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 0.0040 - mse: 0.0090 - val_l
oss: 0.0086 - val_mse: 0.0201
Epoch 17/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0040 - mse: 0.0089 - val_l
oss: 0.0086 - val_mse: 0.0201
Epoch 18/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0040 - mse: 0.0088 - val_l
oss: 0.0085 - val_mse: 0.0200
Epoch 19/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0039 - mse: 0.0087 - val_l
oss: 0.0084 - val_mse: 0.0198
Epoch 20/20
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0038 - mse: 0.0085 - val_l
oss: 0.0084 - val_mse: 0.0196
```

In [7]:
```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.xlabel('Epochs')
plt.ylabel('MSLE Loss')
plt.legend(['loss', 'val_loss'])
plt.show()
```



In [8]:
```
#finding threshold for anomaly and doing predictions
def find_threshold(model, x_train_scaled):
  reconstructions = model.predict(x_train_scaled)
  reconstruction_errors = tf.keras.losses.msle(reconstructions, x_train_s
  threshold = np.mean(reconstruction_errors.numpy()) \
   + np.std(reconstruction_errors.numpy())
  return threshold

def get_predictions(model, x_test_scaled, threshold):
  predictions = model.predict(x_test_scaled)
  errors = tf.keras.losses.msle(predictions, x_test_scaled)
  anomaly_mask = pd.Series(errors) > threshold
  preds = anomaly_mask.map(lambda x: 0.0 if x == True else 1.0)
  return preds

threshold = find_threshold(model, x_train_scaled)
print(f"Threshold: {threshold}")
```

```
73/73 ━━━━━━━━━━━━━━━━━━━━ 0s 958us/step
Threshold: 0.008430090020211022
```

In [9]:
```
#getting accuracy score
predictions = get_predictions(model, x_test_scaled, threshold)
accuracy_score(predictions, y_test)
```

```
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 717us/step
```

Out[9]: 0.955

In [ ]: